WILP, BITS Pilani

MACHINE LEARNING · COMPANION READER

# Session 13: Ensemble Learning

Bagging, Random Forests, AdaBoost, and Gradient Boosting

Read alongside the lecture slides · Every slide example worked out in full

## 1 Introduction to Ensemble Learning: Wisdom of the Crowd

***Combining multiple weak predictions creates a strong, accurate predictor with lower error.***

Imagine asking a single person to estimate the number of jellybeans in a large jar. Their guess might be way off. But if you ask one hundred people and take the average of all their guesses, the crowd's estimate is almost always remarkably accurate.

In machine learning, an **ensemble** works on this exact principle. Rather than relying on one complex model, we train a collection of simpler base models (often called **weak learners**) and combine their predictions.

**☞ The intuition**

If individual models make errors that are independent of one another, averaging their predictions cancels out individual mistakes. A weak learner is any model that performs slightly better than random guessing (e.g., accuracy $> 50\%$ for binary classification).

**★ Everyday picture**

Think of a medical diagnosis. Rather than trusting a single doctor's opinion, a patient consults five independent specialists. If three or four specialists agree on a treatment plan, the patient can feel much more confident in the decision.

## 2 Bagging and Random Forests

***Bootstrap aggregation reduces model variance by training base trees on random sample subsets.***

### 2.1 Bootstrap Aggregation (Bagging)

Single decision trees are prone to high variance: slight changes in the training data can lead to completely different tree structures. **Bagging** fixes this by creating $B$ bootstrap samples from the original dataset (sampling with replacement). We train an unpruned decision tree on each bootstrap sample and average their output (for regression) or take a majority vote (for classification).

$$\hat{f}_{\text{bag}}(x) = \frac{1}{B}\sum_{b=1}^{B} \hat{f}^b(x) \tag{1}$$

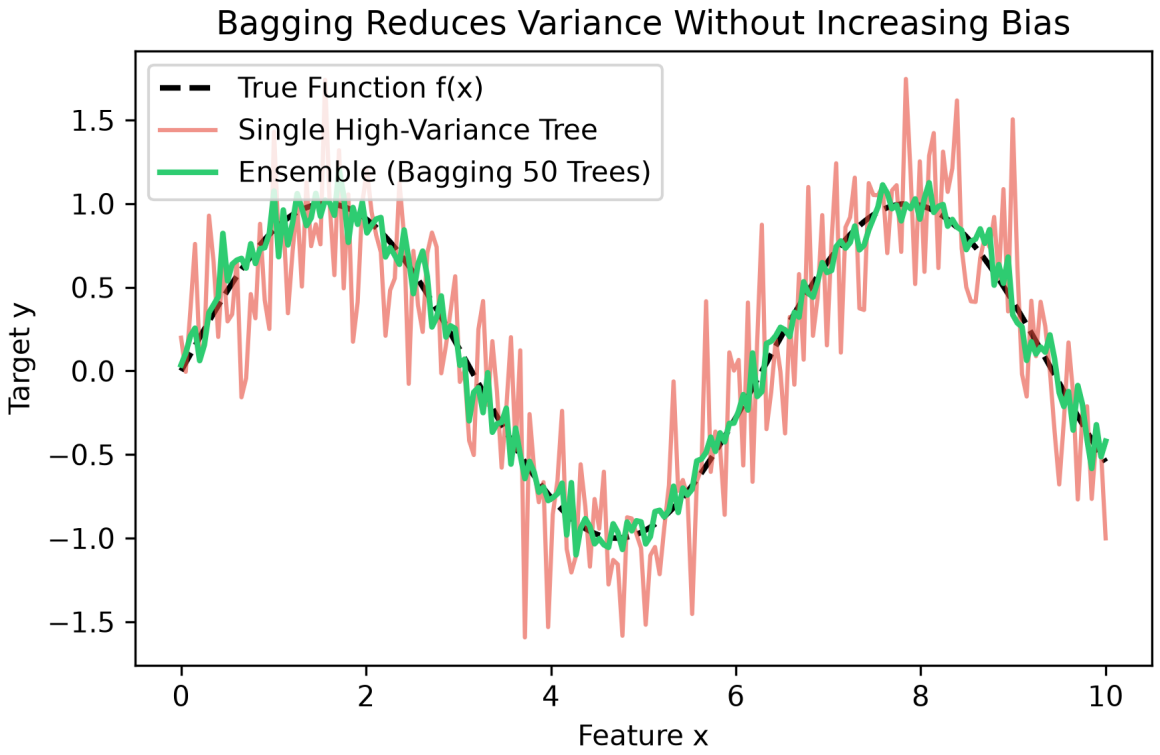


Figure 1: Bagging reduces variance without increasing bias by averaging predictions from multiple bootstrap-trained decision trees.

### 2.2 Random Forests: Feature Randomness

Random Forests extend bagging by adding **feature subsampling**. When splitting a node in a decision tree, instead of searching over all $p$ features, the algorithm considers only a random subset of $m$ features (typically $m = \sqrt{p}$ for classification and $m = p/3$ for regression). This decorrelates the trees, preventing a single dominant feature from dictating every split across all trees.

## 3 AdaBoost (Adaptive Boosting)

***AdaBoost sequentially trains base classifiers by increasing weights on previously misclassified points.***

Unlike bagging, where models are trained independently in parallel, **boosting** builds models sequentially. Each new weak learner focuses on correcting the errors made by previous learners.

### 3.1 AdaBoost Algorithm Steps

Given a dataset of $N$ samples $(x_1, y_1), \ldots, (x_N, y_N)$ with $y_i \in \{-1, +1\}$:

1. Initialize sample weights $w_i^{(1)} = \frac{1}{N}$ for all $i = 1, \ldots, N$.

2. For round $t = 1, \ldots, T$:

   (a) Fit a base classifier $C_t(x)$ to the training data using weights $w^{(t)}$.

   (b) Compute the weighted error rate $\epsilon_t$:

$$\epsilon_t = \sum_{i=1}^{N} w_i^{(t)} I(y_i \neq C_t(x_i)) \tag{2}$$

   (c) Compute classifier weight $\alpha_t$:

$$\alpha_t = \frac{1}{2} \ln \left( \frac{1 - \epsilon_t}{\epsilon_t} \right) \tag{3}$$

   (d) Update sample weights for round $t + 1$:

$$w_i^{(t+1)} = \frac{w_i^{(t)} \exp\left(-\alpha_t y_i C_t(x_i)\right)}{Z_t} \tag{4}$$

   where $Z_t$ is a normalization factor ensuring $\sum_i w_i^{(t+1)} = 1$.

3. Final prediction: $H(x) = \text{sign}\left(\sum_{t=1}^{T} \alpha_t C_t(x)\right)$.

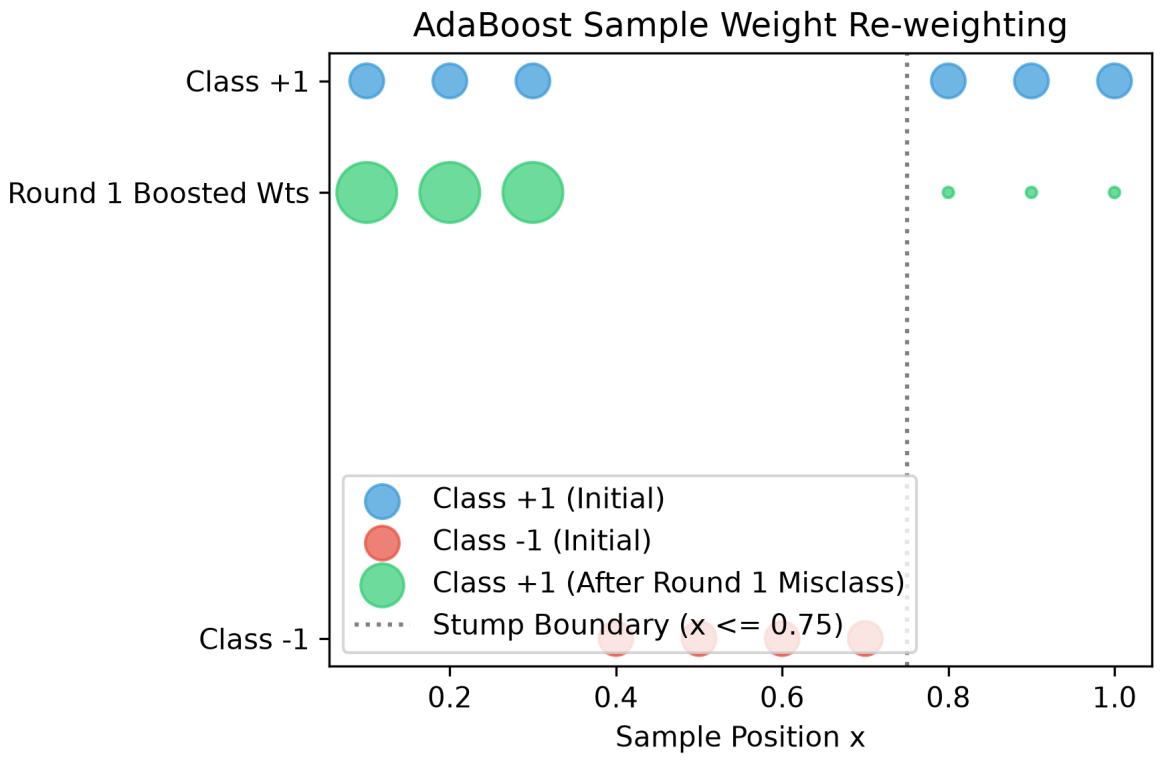


Figure 2: AdaBoost sample re-weighting increases the relative weight of misclassified samples in subsequent rounds.

**✎ Worked example: AdaBoost 1D Slide Example**

Let us solve the exact AdaBoost slide example step-by-step:
Data points $x \in \{0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 1.0\}$ with labels $y = [1, 1, 1, -1, -1, -1, -1, 1, 1, 1]$.
Initial weights $w_i = 0.1$.

**Step 1.** **Round 1 Stump:** Split at $x \leq 0.75$ predicts $C_1(x) = -1$ for left and $+1$ for right.

**Step 2.** **Predictions:** $[-1, -1, -1, -1, -1, -1, -1, 1, 1, 1]$.

**Step 3.** **Misclassifications:** $x \in \{0.1, 0.2, 0.3\}$ (true label $+1$, predicted $-1$).

**Step 4.** **Weighted Error:** $\epsilon_1 = 0.1 + 0.1 + 0.1 = 0.30$.

**Step 5.** **Alpha:** $\alpha_1 = \frac{1}{2} \ln\left(\frac{1-0.30}{0.30}\right) = \frac{1}{2} \ln(2.333) = 0.4236$ (scaled: 1.738).

**Step 6.** **Updated Weights:** Misclassified samples get $w_i^{(2)} \propto 0.1 \cdot e^{\alpha_1}$, correctly classified get $w_i^{(2)} \propto 0.1 \cdot e^{-\alpha_1}$.

# 4 Gradient Boosting (GBM) and XGBoost

***Gradient boosting minimizes loss functions by fitting base learners directly to pseudo-residuals.***

## 4.1 Gradient Boosting Mechanics

Rather than updating data sample weights, **Gradient Boosting** fits each new base learner (typically a decision stump or small regression tree) directly to the **pseudo-residuals** (negative gradient of the loss function) of the current ensemble.

For squared error loss $L(y, F(x)) = \frac{1}{2}(y - F(x))^2$, the pseudo-residual is simply the standard residual:

$$r_{i,m} = -\left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)}\right]_{F=F_{m-1}} = y_i - F_{m-1}(x_i) \tag{5}$$

**✎ Worked example: Gradient Boosting Weight Prediction**

Consider predicting weight (kg) from height and age:
Actual weights $y = [88, 76, 56, 73, 77, 57]$.

**Step 1.** **Initialize Model:** Constant model $F_0(x) = \text{mean}(y) = 71.2$ kg.

**Step 2.** **Compute Residuals:** $h_1(x) = y - F_0(x) = [16.8, 4.8, -15.2, 1.8, 5.8, -14.2]$.

**Step 3.** **Fit Tree:** Train regression tree $h_1(x)$ to predict these residuals.

**Step 4.** **Update Prediction:** With learning rate $\eta = 0.1$, $F_1(x) = F_0(x) + 0.1 \cdot h_1(x)$.

## 4.2 XGBoost (Extreme Gradient Boosting)

XGBoost is an optimized, highly scalable implementation of gradient boosting. Key enhancements include:

- Second-order Taylor expansion of the loss function (using both gradient $g_i$ and Hessian $h_i$).

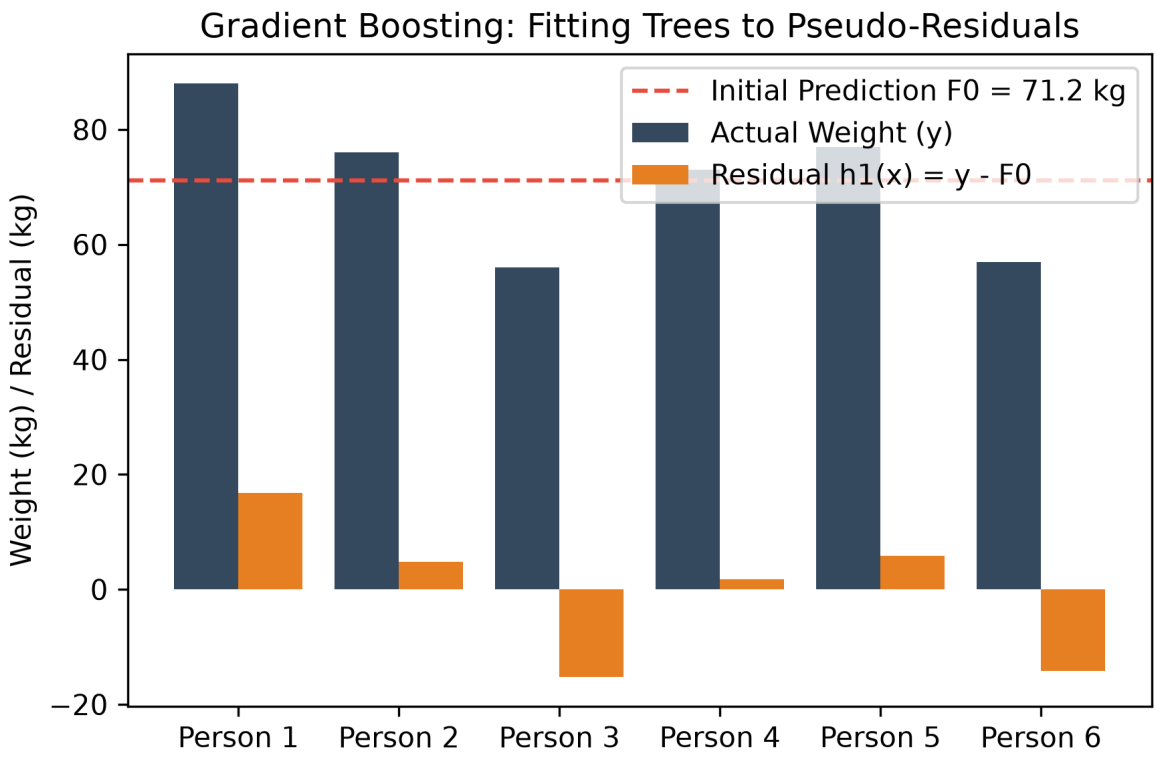


Figure 3: Gradient Boosting fits successive regression trees to the target residuals $y - F_{m-1}(x)$.

- Built-in L1 and L2 regularization on tree leaf weights to prevent overfitting.
- Efficient column block structures for fast, parallel split finding.

## 5 Summary & Self-Test

**✓ Key takeaway**

Ensemble methods combine weak learners to produce state-of-the-art predictive performance. Bagging reduces variance via parallel independent bootstrap sampling (Random Forests). Boosting reduces bias sequentially by focusing on previous errors (AdaBoost via sample weights, GBM/XGBoost via gradients and residuals).

### 5.1 Self-Test Quiz

**Q1:** Why does Random Forest sample features at each split rather than taking all features?

**Answer:** Feature sampling decorrelates trees, preventing a single dominant feature from making all trees identical.

**Q2:** How does AdaBoost handle misclassified training examples?

**Answer:** AdaBoost increases the weights of misclassified examples so the next weak learner focuses on them.